

Sistemas Operativos

1° Rec 2° Parcial 2C2024 – Resolución

Aclaración: La mayoría de las preguntas o ejercicios no tienen una única solución. Por lo tanto, si una solución particular no es similar a la expuesta aquí, no significa necesariamente que la misma sea incorrecta. Ante cualquier duda, consultar con el/la docente del curso.

Teoría

1.

	Contigua	Enlazada	Indexada
Fragmentación	Interna en el último bloque y externa	Interna en el último bloque	Interna en el bloque índice y en el último bloque de datos
Flexibilidad crecimiento de archivos	Es poco flexible ya que requiere bloques contiguos al archivo	El más flexible, cualquier bloque se puede enlazar	Se puede indexar cualquier bloque pero limita el tamaño máximo
Acceso directo a bloque	Puede acceder directamente a cualquier bloque	Debe acceder a los bloques de forma secuencial	Accede al bloque índice y luego al bloque requerido

2. En memoria real se tiene: número de frame y bit de validez.
En memoria virtual se cambia el bit de validez por bit de presencia, se agrega además el bit de modificado y otros datos necesarios para el reemplazo de páginas como pueden ser instante de carga, referencia y/o bit de uso.
3. Si se tiene reemplazo local de páginas, el thrashing ocurre localmente en uno o más procesos en específico a los que se les debe asignar más frames.
En reemplazo global se puede solucionar reduciendo el grado de multiprogramación.
4. Por un lado Verdadero, al tener asignación fija implica necesariamente que el reemplazo es local. Cambiando la asignación a variable, permitiría reemplazo global.
Podría también justificarse Falso ya que la asignación variable se puede mantener con reemplazo local.
 - b. Falso, aumenta la cantidad de acceso a memoria ya que se requiere acceder a varios niveles de tablas en memoria para realizar la traducción.
5.
 - a. Verdadero, contiene todos los permisos de todos los usuarios para cada archivo, sin embargo su mantenimiento es más costoso por ser una única estructura global (tamaño, sincronización, etc.)
 - b. Falso, sus punteros apuntan a uno o más bloques cuyo contenido es la ruta al archivo original.

Práctica

1.

- a) Si la dirección física 1AEF0h hace referencia al frame 6 podemos deducir cómo se interpreta la misma: 0001 10|10 1110 1111 0000
Para que haga referencia al frame 6 la dirección se divide entonces en 14 bits para offset y el resto para número de frame.
Luego, para saber qué dirección lógica la generó vemos en la tabla que la página que está presente en ese frame es la página 2.
Por lo tanto, la dirección lógica quedaría: 10|10 1110 1111 0000 -> **AEF0h**
- b) COFFh -> 11|00 0000 1111 1111 -> página 3 no está presente, page fault.
Si genera la dirección física 2COFF => 0010 11|00 0000 1111 1111 => frame 11. Es la página más recientemente referenciada y tiene bits U=1 y M=0. Por lo tanto:
LRU no pudo haber sido utilizado.
Clock modificado seguro habría reemplazado ese frame ya que tiene sus bits U=1 y M=1.

2.

Disco de 1 TiB = 2^{30}

Partición 1 = 512 GiB = 2^{29} Ext Inodo = 5 PD + 2PIS + 1PID. Punteros 32 bits y bloques de 2048 bytes $\Rightarrow$ CPB = $2048 / 4 = 512$ Punteros por bloque

Partición 2 = 256 GiB = 2^{28} FAT16 = 16 bits para direccionar y TB=1024 bytes

Partición 2 = 256 GiB = 2^{28} FAT32 = 28 bits para direccionar y TB=2048 bytes

a) Partición 1:

TMax Teorico = $2^{32} * 2^{11} = 2^{43} = 8 \text{ TiB}$

TMax Real = **512GiB**

Partición 2:

TMax Teorico = $2^{16} * 2^{10} = 2^{26} = 64 \text{ MiB}$

TMax Real = **64 MiB**

Partición 3:

TMax Teorico = $2^{28} * 2^{11} = 2^{39} = 512\text{GiB}$

TMax Real = **256 GiB**

b) Archivo = 5.769.216 bytes = 5,502MiB y TB = 2048 Bytes

TMax Arch = $(5 + (2 * 512) + (1 * (512^2))) * 2048 = 514,0098 \text{ MiB}$

Este FS tolera archivos de aproximadamente 514 MiB entra en dicha Partición.

Total de Bloques de Datos por leer son: 5.769.216 bytes/2048 Bytes = **2817**

$2817 - 512 \text{ Bloques 1PIS} - 512 \text{ Bloques 2Pis} = 1793 \text{ bloque de PID}$

$\text{PID} = 1793 / 512 = 3,5019 = \text{Son 5 accesos PID}$

Total accesos = 2817 BD + 2 PIS + 5PID = 2824 accesos.

c) Se puede migrar a ambas particiones dado que el archivo pesa 5,502 MiB

Partición 2:

$5.769.216 \text{ bytes} / 1024 \text{ Bytes} = 5634 \text{ Bloques}$

Para leer el archivo necesito **2824 accesos**

Bloque a escribir son $5634 + 1 \text{ bloque directorio}$

Para Escribir son necesarios **5635 accesos**

Partición 3:

Para leer el archivo necesito **2824 accesos**

Bloques a escribir son $2817 + 1 \text{ bloque de directorio.}$

Para Escribir el archivo necesito **2818 accesos**

3.

a) Representamos el estado actual de la memoria para saber qué espacios quedan libres.

PA-S0 256	128	PB-S0 128	64	PB-S1 128	64	PA-S1 256
-----------	-----	-----------	----	-----------	----	-----------

Vemos que de los dos segmentos de 100 KiB solamente podríamos asignar uno de ellos antes de compactar. Se podría asignar uno de ellos y compactar luego o compactar primero y luego asignar ambos de forma contigua, lo resolveremos de esta última forma.

Hay distintas formas de realizar la compactación, tomaremos la forma que menos segmentos requiera mover, por lo que moveremos los segmentos de PB para estar contiguos a PA, por lo tanto la tabla de segmentos de PA nos quedará igual y tendremos un espacio libre de 256 KiB.

PA-S0 256	PB-S0 128	PB-S1 128	256	PA-S1 256
-----------	-----------	-----------	-----	-----------

Al asignar los dos segmentos nos quedaría un **espacio libre de 56 KiB** y las tablas:

Tabla Segmentos PA

Base	Tamaño
0	256
768	256

Tabla Segmentos PB

Base	Tamaño
256	128
384	128

Tabla Segmentos PC

Base	Tamaño
512	100
612	100

Nota: Todos los valores están expresados en KiB

Cabe destacar que las bases de los segmentos pueden variar según cómo se haya planteado la compactación de la memoria.

b) Tam tabla FAT = $2^{16} * 2^1 = 2^{17}$ B = **128 KiB**

Luciano está en lo correcto, no entraría en el único hueco disponible de 56 KiB.

Al estar limitado por el disco de 8 MiB, el FS ahora tendría:

$8\text{MiB}/1\text{KiB} = 2^{23}/2^{10} = 2^{13}$ bloques

=> Tam tabla FAT = $2^{13} * 2^1 = 2^{14}$ B = **16 KiB**

La propuesta de Luciano cumpliría el objetivo ya que podría alojar la tabla en el espacio libre.